MODULE 09 · 22 MIN

Skills, Hooks, MCP & Multi-Agent

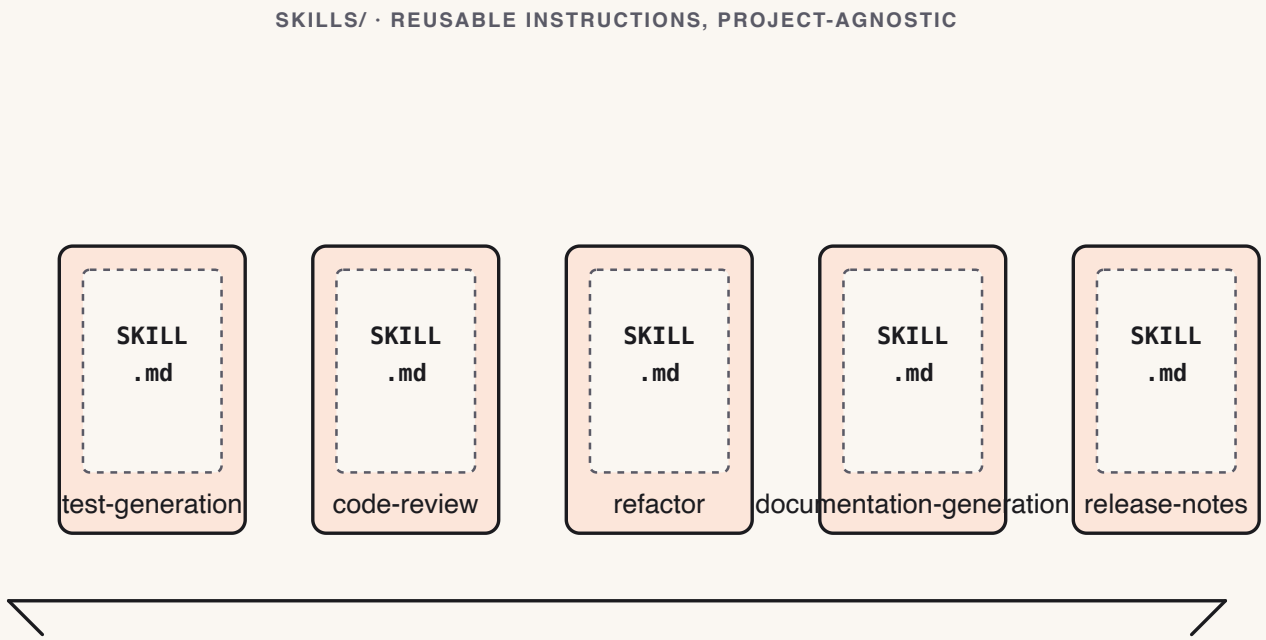
Stop re-typing workflows. Package them. This is agentic engineering.

Theory · Four pillars of agentic engineering (4 min)

1. **Skills** — packaged workflows at `skills/<name>/SKILL.md`, invoked with `/<name>`. Your carry-out from today.
2. **Hooks** — shell commands that fire before/after Claude actions (format, lint, deny dangerous commands).
3. **MCP** — open connectors to issue trackers, docs, chat, internal tools, read as first-class context.
4. **Multi-agent** — a lead agent fans work out to workers in separate worktrees.

> Skills are the unit you'll reuse most. The rest make Claude a teammate, not a chatbot.

The four power-ups



Skills · Hooks · MCP · Multi-agent — Skills are the one you'll reuse most.

Reference · Hooks & MCP

Hooks — `.claude/hooks.json`:

- `post-edit` → `npx prettier --write` (auto-format).
- `pre-bash` → deny `rm -rf` (guardrail).
- `pre-commit` → run tests (no broken commits).

MCP (Model Context Protocol) — connect, don't paste:

- Jira / Linear / GitHub · Drive / Notion · Slack.
- **Least privilege**: read scope unless you truly need write.

Reference · Multi-agent & common mistakes

Multi-agent fan-out — lead splits a feature across workers (backend / frontend / tests) in separate worktrees. **Don't** use it for tasks < 30 min, shared mutable state, or non-independent sub-tasks.

Common mistakes

- Vague skill body ("do a good job").
- Project-specific paths/versions in a skill (won't carry over).
- Skipping the **Worked example** (reviewers can't tell if it works).
- Fanning out 4 agents on a 10-minute task.

Live demo · Author a Skill in 4 minutes (5 min)

1. Open `skills/code-review/SKILL.md`; read its H2 headers aloud.
2. Paste the drafting prompt:

```
Draft a SKILL.md named "score-candidates" using the same H2 sections as
code-review/SKILL.md. Purpose: score 3 diffs on Correctness, Simplicity, Fit.
```

3. Invoke it: `/score-candidates` on three diffs.
4. Observe Claude produce output matching the skill's **Outputs** section.

Success signal: the new skill runs and its output matches its own spec — no extra prompting.

Live demo · Connect GitHub over MCP (4 min)

"Connect, don't paste." Let Claude read GitHub directly instead of copying issue text into chat.

1. Add the connector with a **fine-grained, read-scoped** token:

```
claude mcp add --transport http github https://api.githubcopilot.com/mcp/ \
  --header "Authorization: Bearer $GITHUB_PAT"
```

2. Verify + authenticate: run `/mcp` (status should read **connected**).
3. Ask a question that needs the live repo — paste:

```
Show me all open PRs assigned to me, then summarize what PR #456 changes.
```

Success signal: real PR data Claude could only get by reading the connector — zero copy-paste.

▶ Your turn · Author a reusable Skill (10 min)

Exercise: `exercises/part-09/README.md`

Write one **project-agnostic** skill for a workflow you'll repeat (e.g. `commit-and-pr`, `screenshot-diff`, `regression-postmortem`):

- Valid frontmatter (`name`, `description`); all **6 H2 sections** in order; body ≤ 80 lines.
- **No** project-specific paths, filenames, or versions.

Deliverables: `module-09/skill/SKILL.md` · `module-09/invocation.md` (one real invocation + output).

Stretch: add a `hooks.json`, sketch an `mcp-brief.md`, or fan a task out to two sub-agents.

Success signal: one real invocation whose output matches the skill's **Outputs** section.

✓ Done & next (1 min)

Definition of done

- ☐ [ ] Valid frontmatter; all 6 H2 sections in order; body ≤ 80 lines.
- ☐ [ ] No project-specific paths/versions in the body.
- ☐ [ ] One real invocation with output matching the **Outputs** spec.

Next — we decide if any of today's projects is actually ready to ship.
Module 10 — Production Readiness.